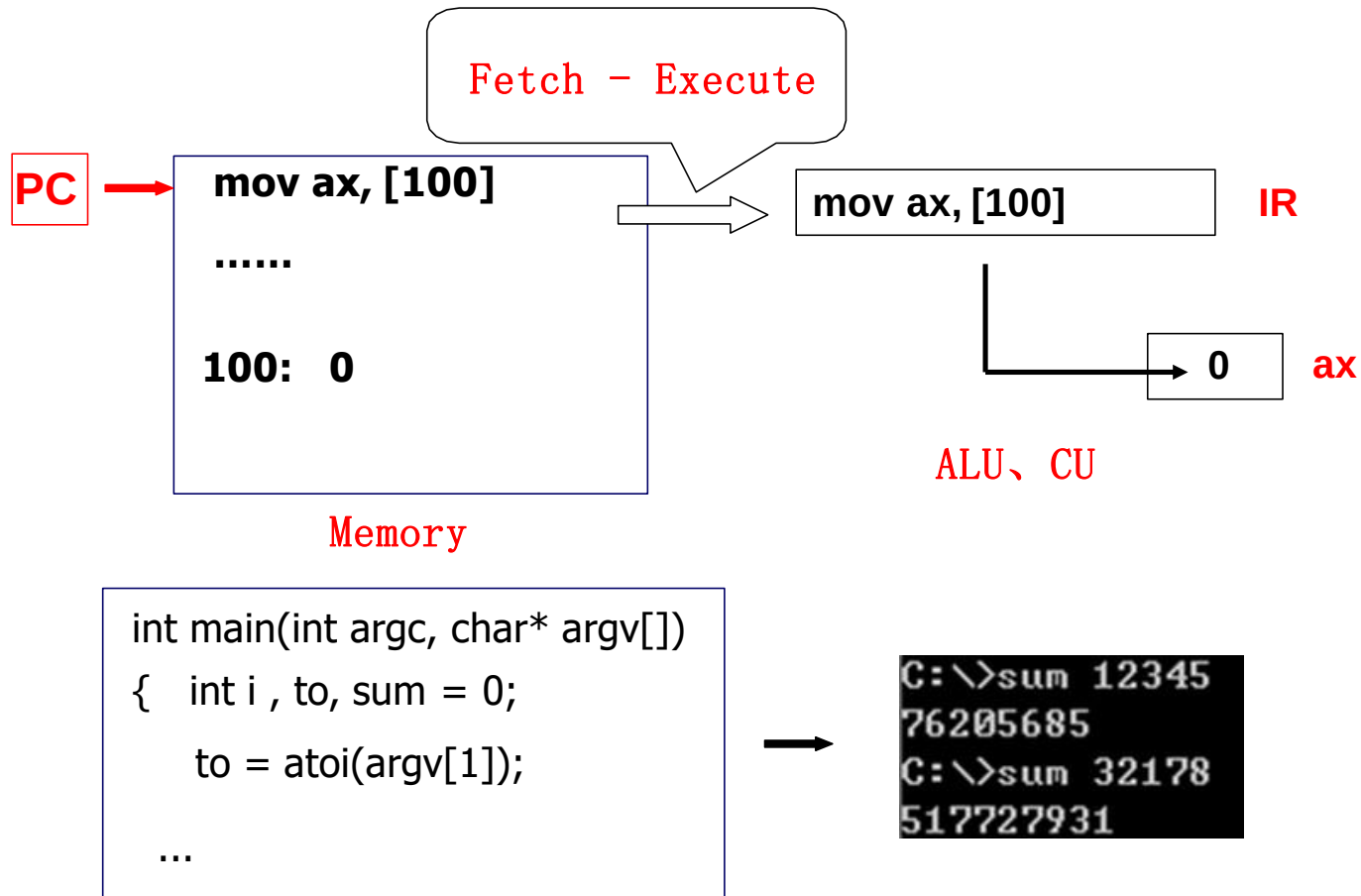


Computer Architecture

内存和虚拟内存 ***Memory and Virtual Memory***

How does memory work?

Instruction Fetch - Execute



Memory usage:

Place the program into memory, and the PC points to the starting address of the program.

Program is loaded into the memory

```
int main(int argc, char* argv[])  
{ ...
```

```
.text  
_entry: //Entry Address  
    call _main  
    call _exit  
_main:  
    ... ret
```

```
_entry: //Entry Address  
    call 40  
    call xx  
_main: //Offset is 40  
    ...
```

Memory

```
...  
40 _main: mov [300], 0  
...  
call xx  
call 40
```

IP → 0

Memory is now available, but...

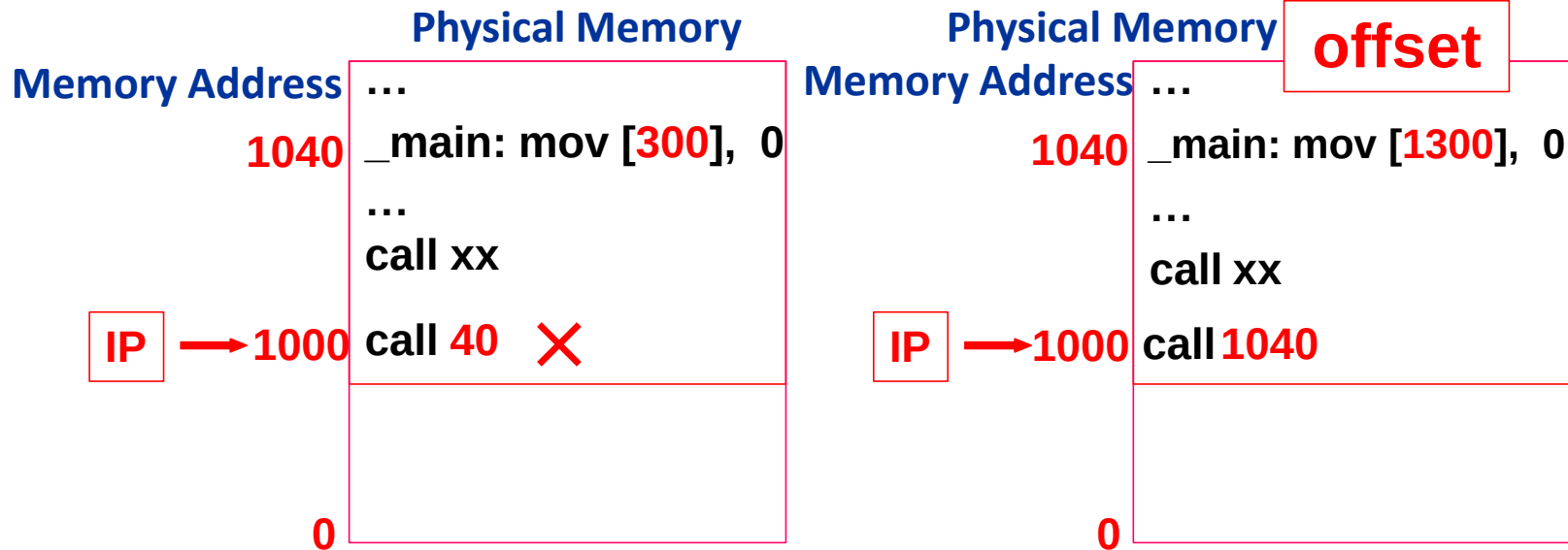
Memory

```
...  
1040 _main: mov [300], 0  
...  
call xx  
call 40 X
```

IP → 1000

0

Memory Relocation



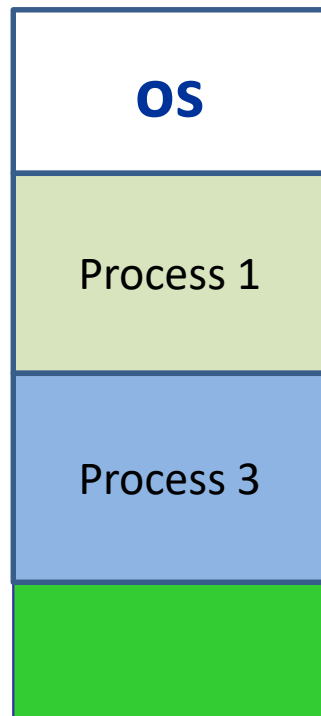
When is relocation completed? **At compile time** **At load time**

- A program that is relocated at compile time can only be placed at a fixed position in memory
- A program that is relocated at load time cannot be moved once it is loaded into memory

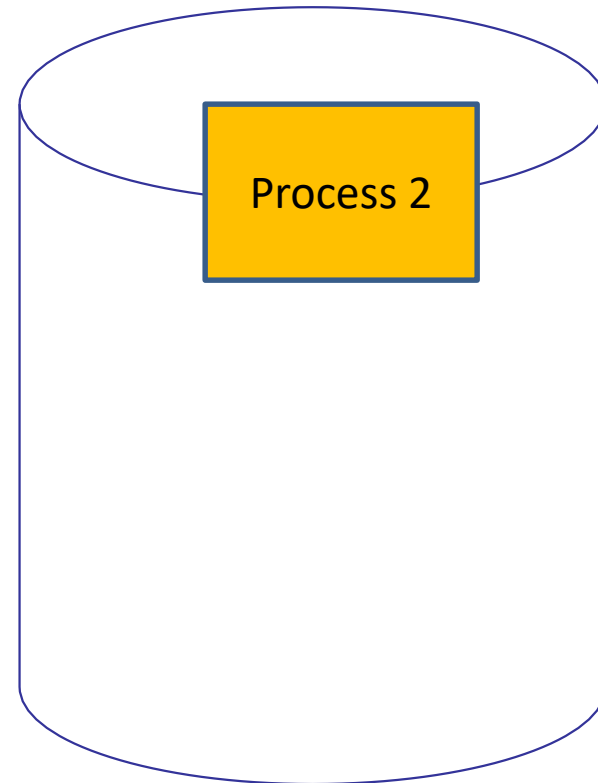
Memory Relocation

Swap:

The program needs to be moved even after it is loaded



Memory

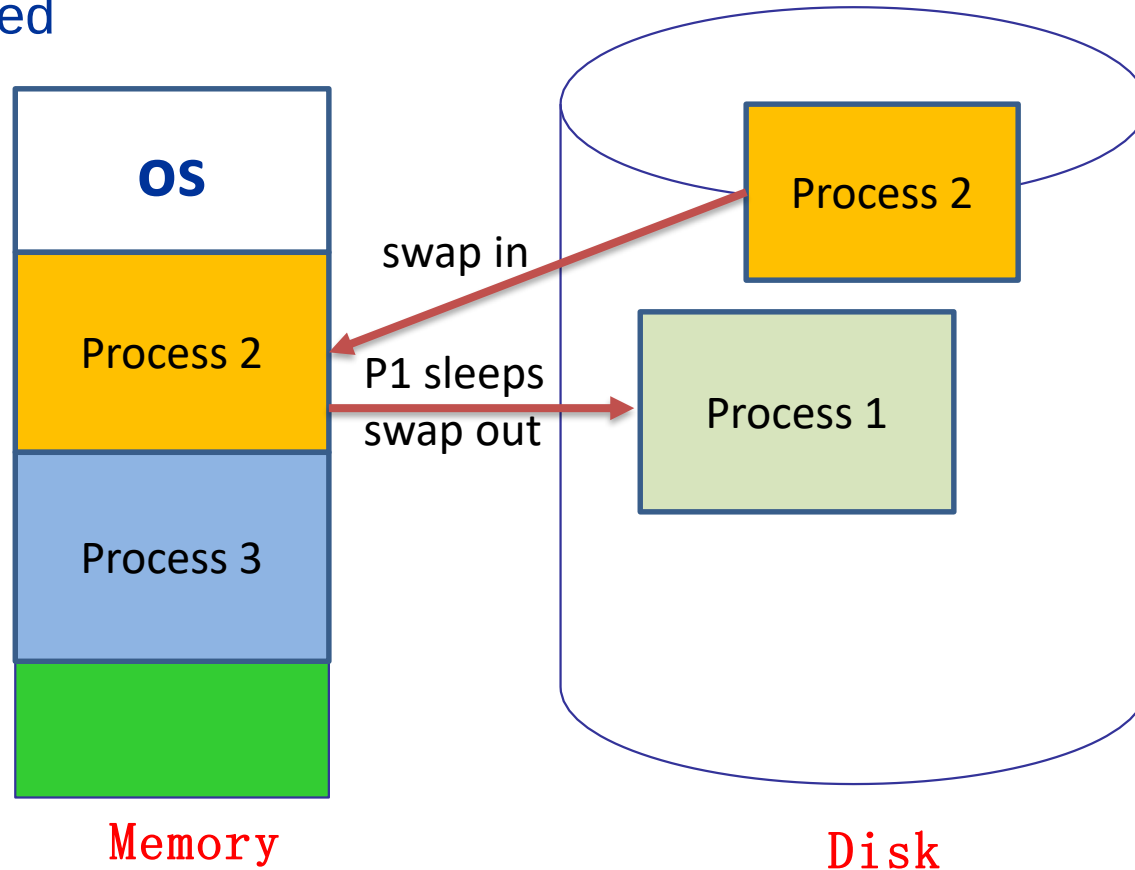


Disk

Memory Relocation

Swap:

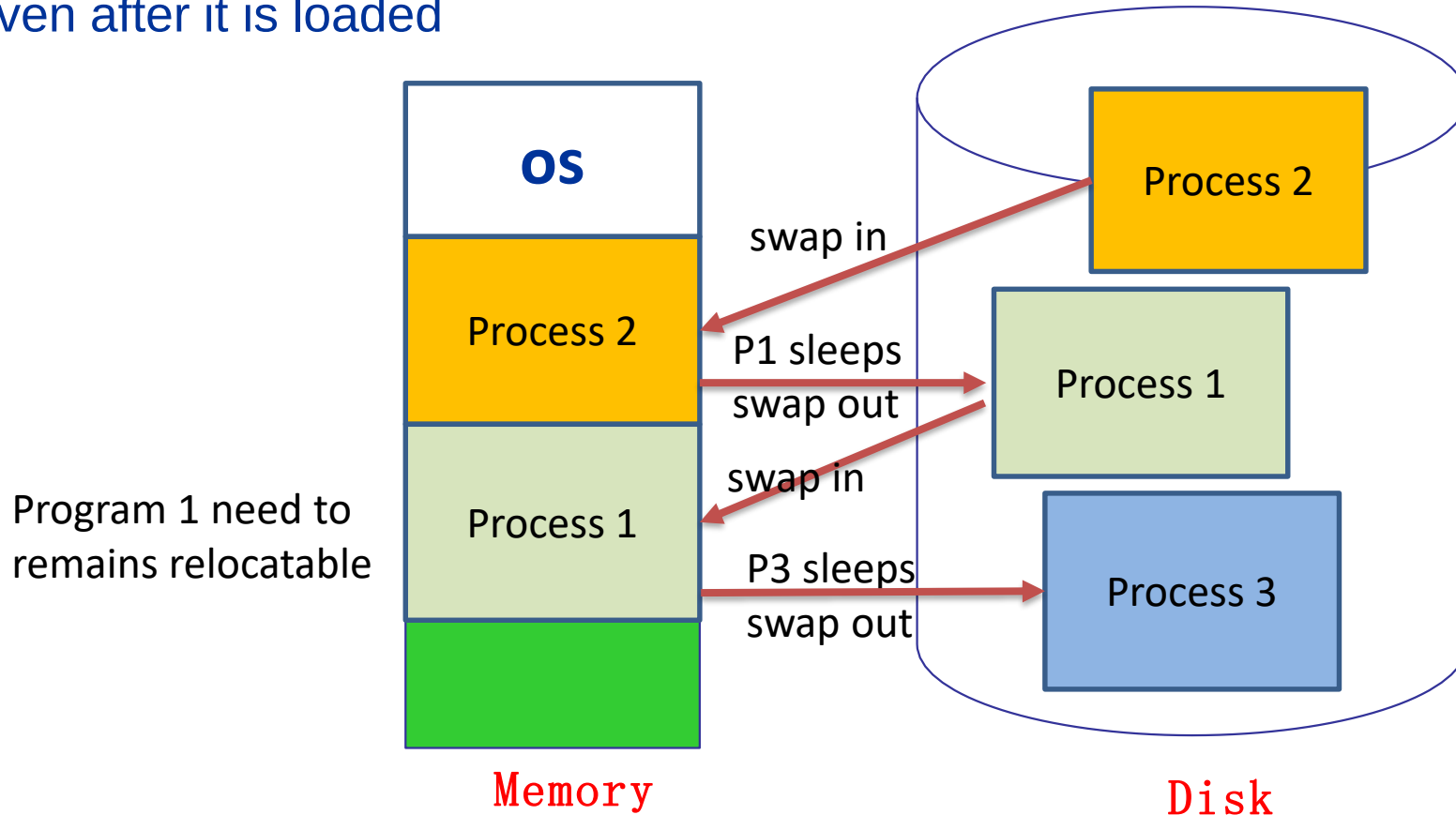
The program needs to be moved even after it is loaded



Memory Relocation

Swap:

The program needs to be moved even after it is loaded

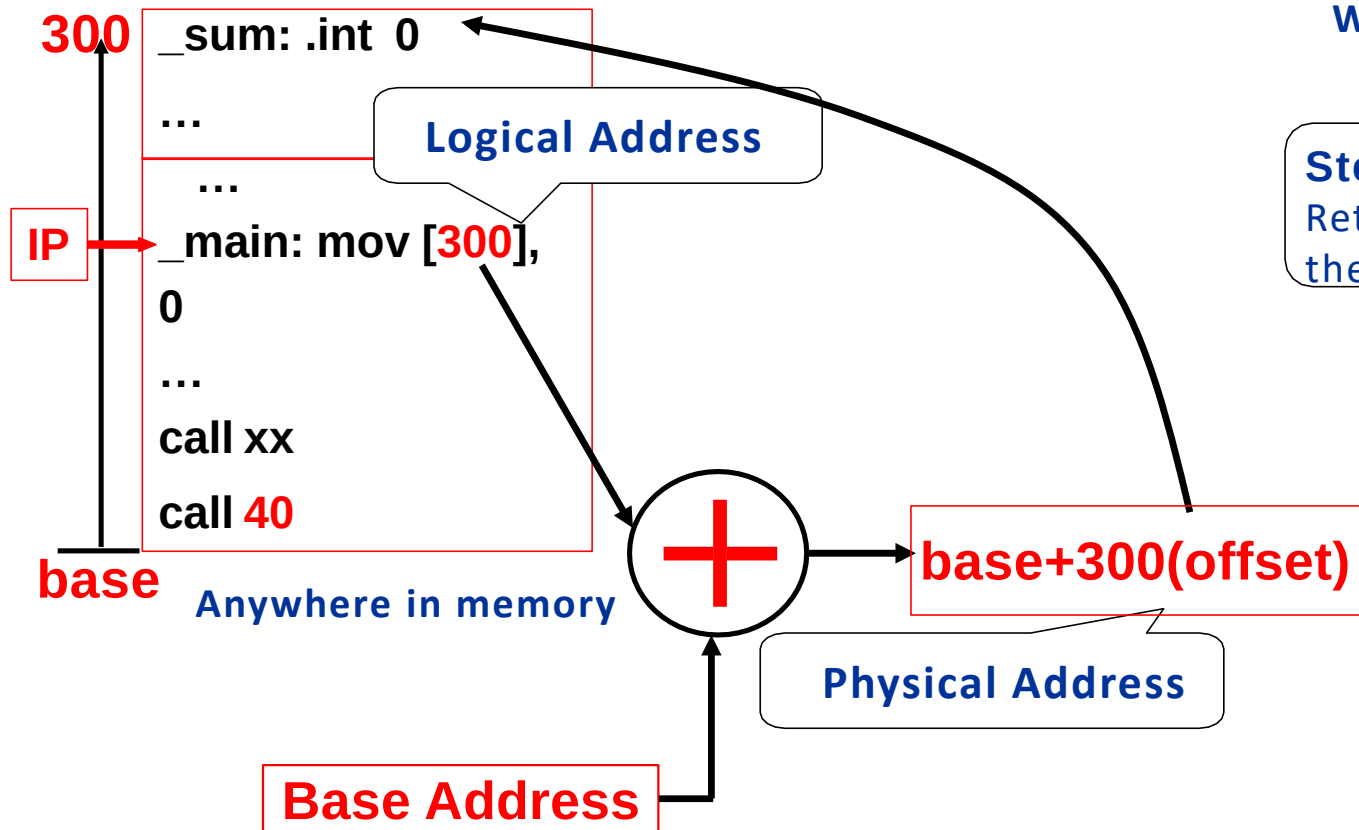


Memory Relocation

Relocation is performed when executing each instruction

Translate the logical address to a physical address for each instruction execution:

Address Translation



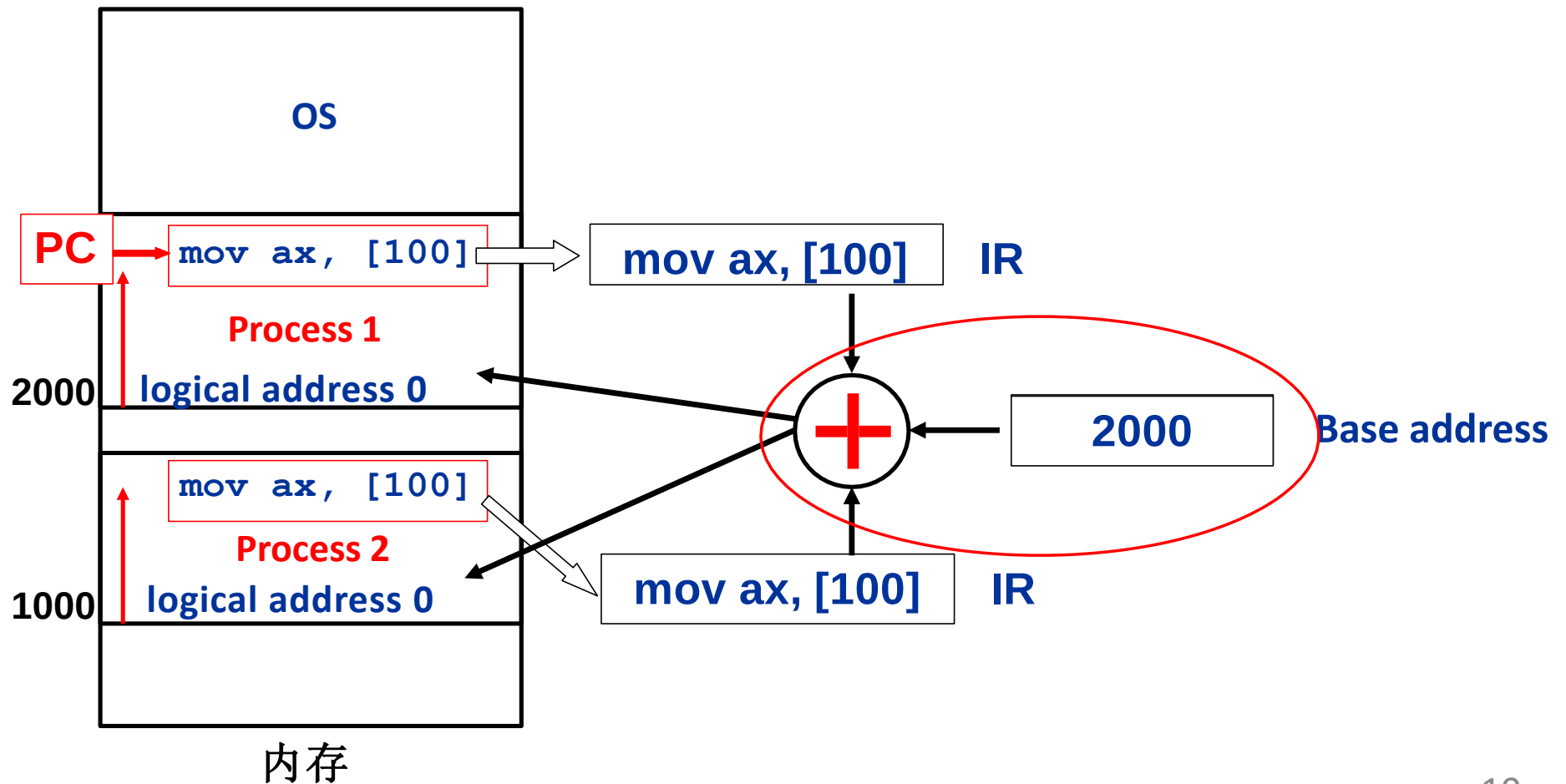
Each process has its own base address.

Where are they stored?

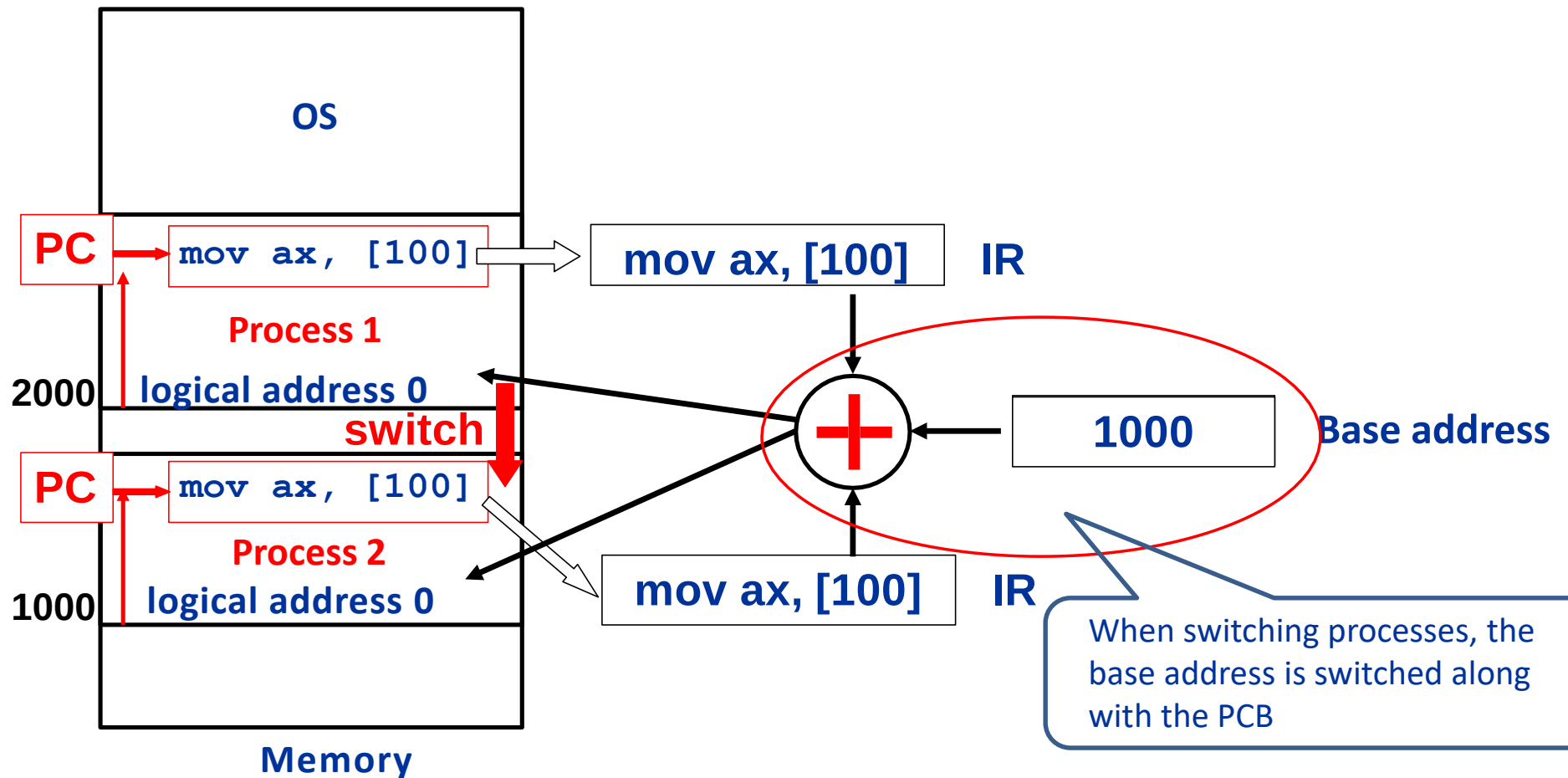
PCB

Step 1(instr. exec.):
Retrieve the base address from the PCB

Memory Relocation



Memory Relocation



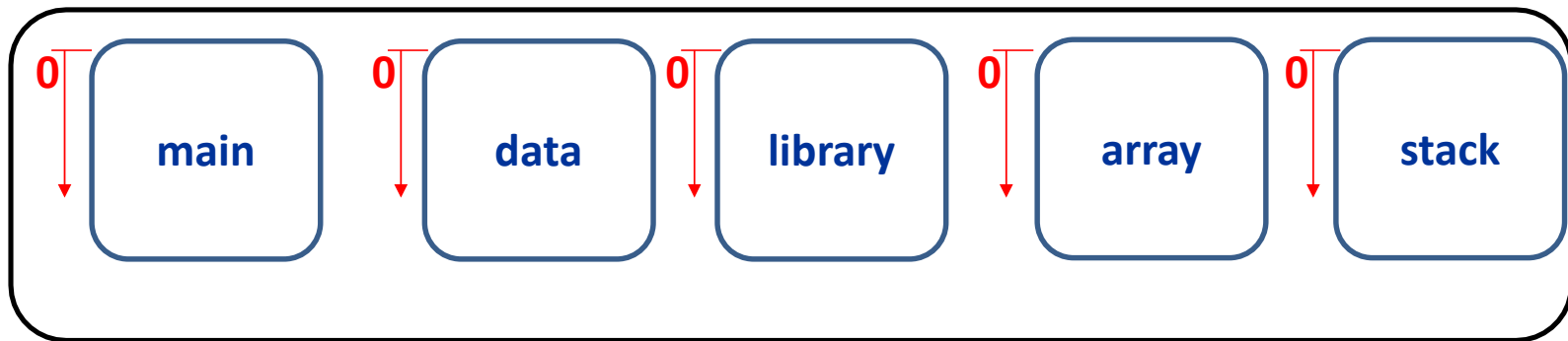
分段
Segmentation

***Will the entire program be loaded into memory
all at once?***

The program from the programmer's perspective

Consists of several segments, each with its own characteristics and purposes:

the code segment is read-only, the code/data segment does not dynamically grow...



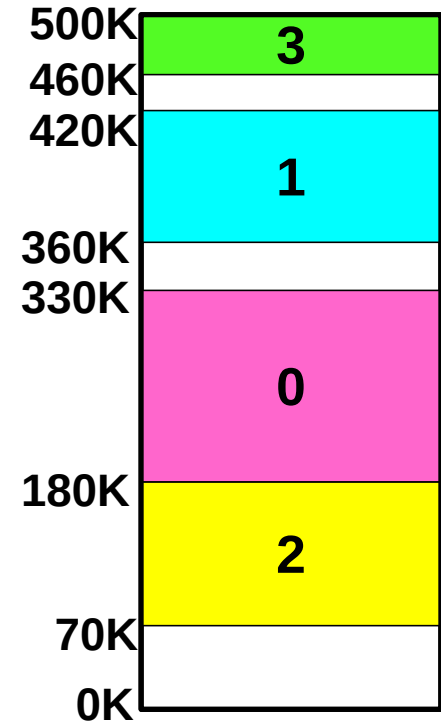
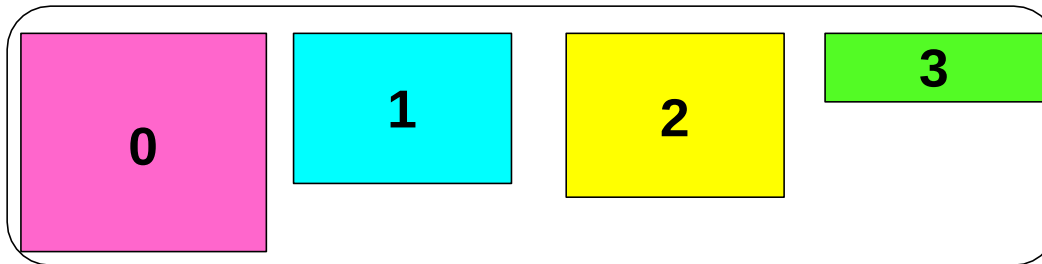
A program in the eyes of a programmer

Programmers can consider each segment independently (Divide&Conquer)

Locate instructions(data): <seg num , offset>

`mov [es:bx], ax`

Program segments are placed into memory



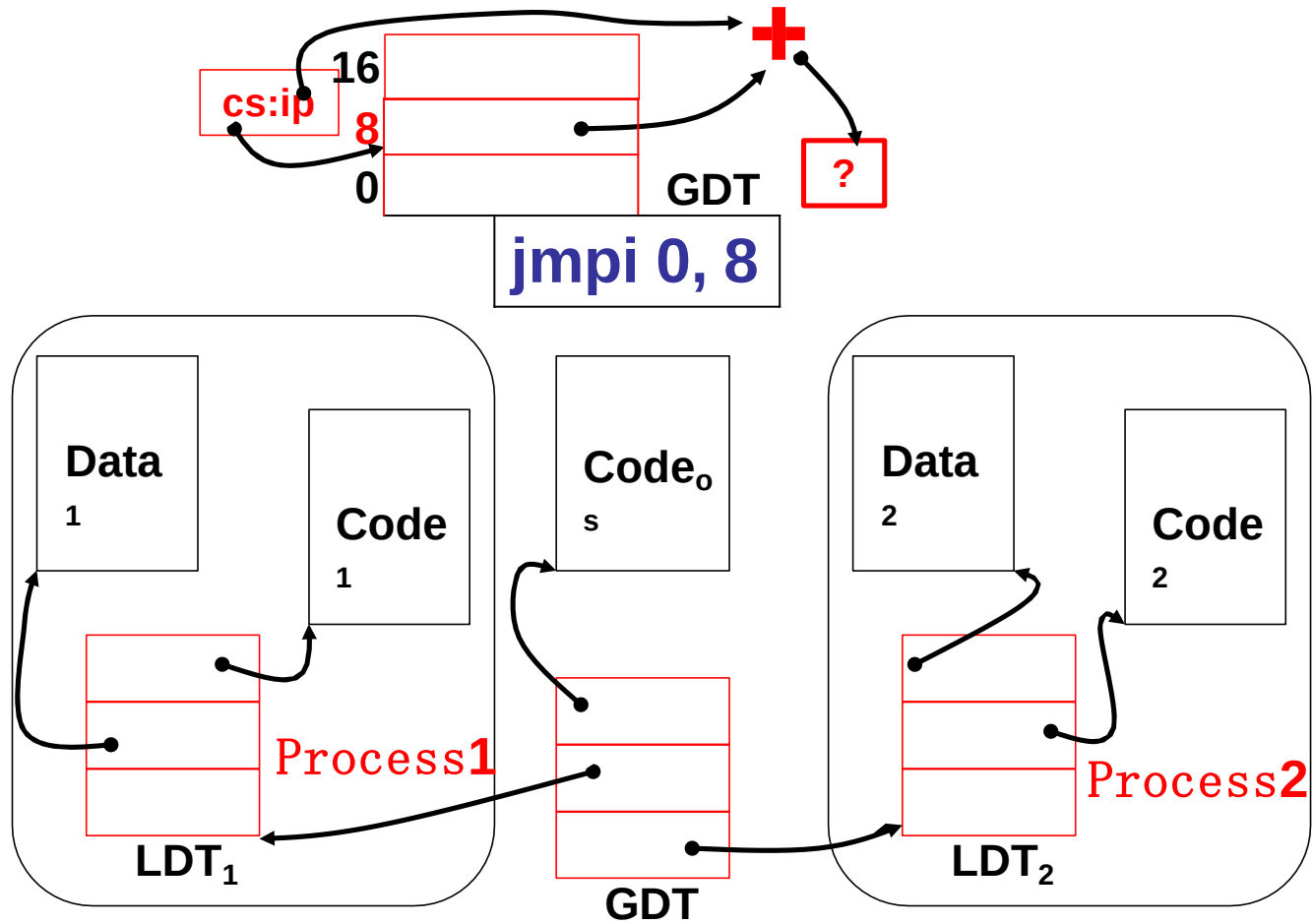
What happens when runtime relocation is performed?

mov [DS:100], %eax jmp 100, CS

LDT	Seg#	Base Addr.	Size	Priv.
	0	180K	150K	R
	1	360K	60K	R/W
	2	70K	110K	R/W
	3	460K	40K	R

Question: Assuming DS=1, CS=0, what would the relocation of the above two instructions be at runtime?

GDT and LDT

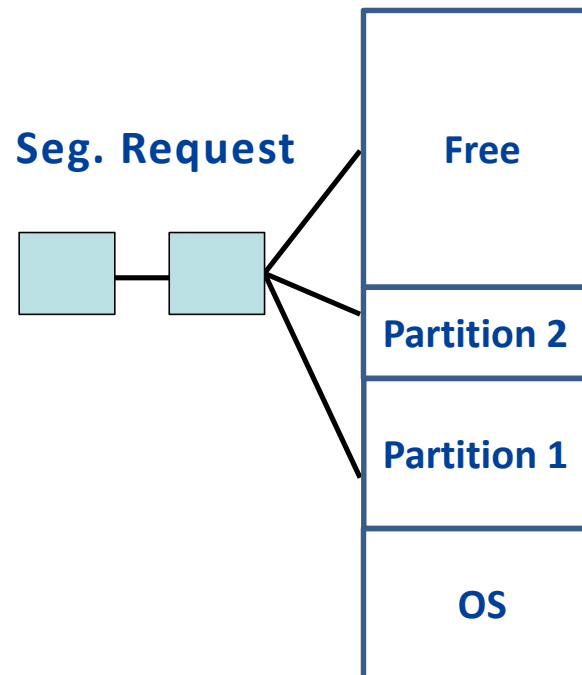
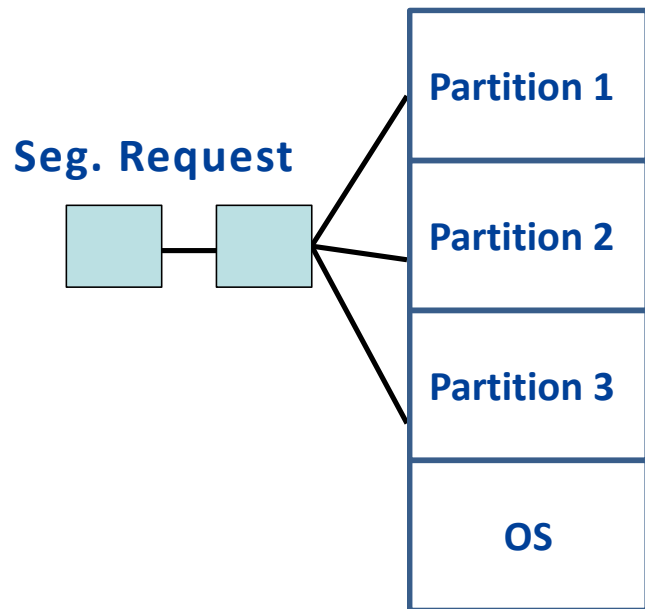


分区 *Partition*

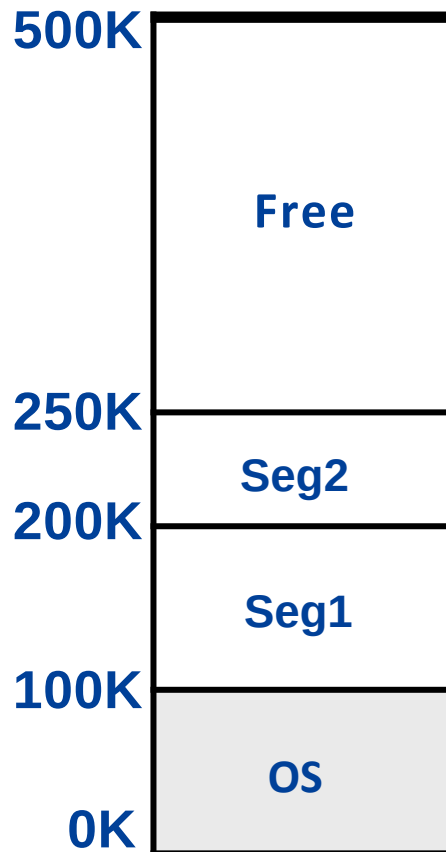
Fixed Partitioning and Variable Partitioning

A loaf of bread, and a group of children come to eat it, what do you do?

- Equal division, the operating system divides the memory into k equal partitions during initialization.
- Children come in different sizes, and so do segments; the requirements are not fixed



Management process of variable partitioning



Free partition table

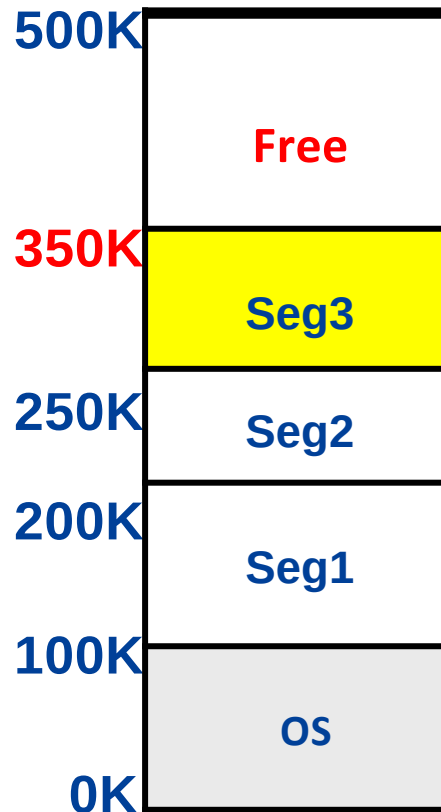
Star. Addr	Size
250K	250K

Allocated partition table

Star. Addr	Size	Sign
0K	100K	OS
100K	100K	Seg1
200K	50K	Seg2

P1: Request allocation

Segment memory request: reqSize = 100K



Free partition table

Star. Addr	Size
350K	150K

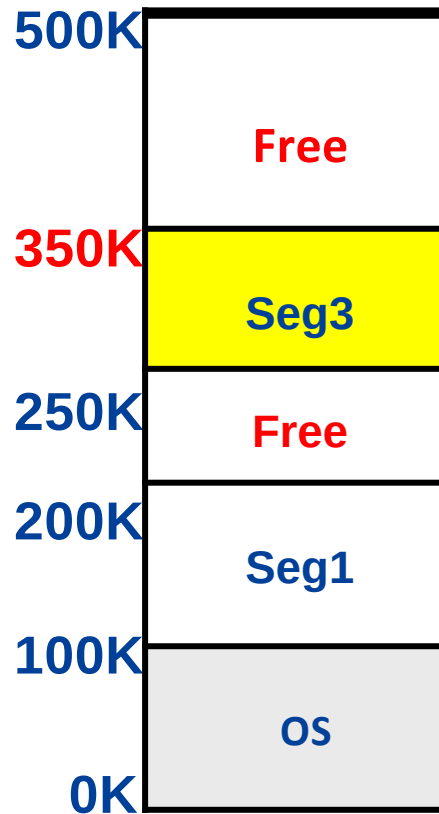
Allocated partition table

Star. Addr	Size	Sign
0K	100K	OS
100K	100K	Seg1
200K	50K	Seg2
250K	350K	Seg3

P2: Release memory

Segment 2 is no longer needed

Release the memory



Free partition table

Star. Addr	Size
350K	150K
200K	50K

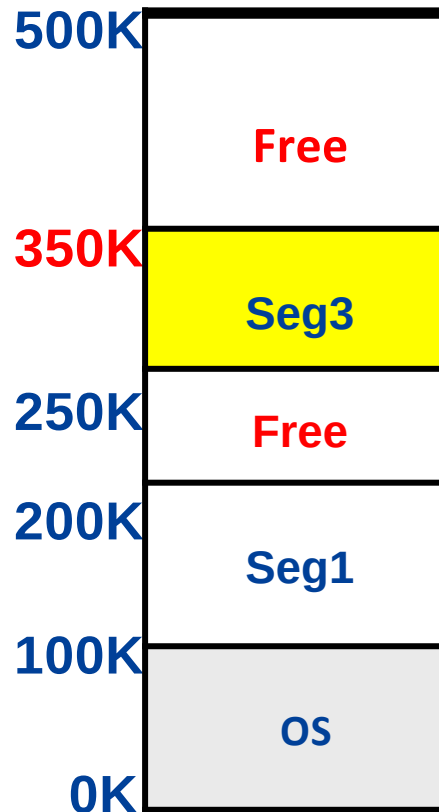
Allocated partition table

Star. Addr	Size	Sign
0K	100K	OS
100K	100K	Seg1
200K	50K	Seg2

P3: Apply for memory again

Another segment requests memory:

reqSize=40K, what should be done?



Free partition table

Star. Addr	Size
350K	150K
200K	50K

There are 2 free partitions, which one should be chosen?

- First fit: (350, 150)
- Best fit: (200, 50)
- Worst fit: (350, 150)

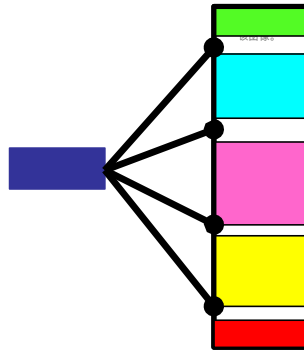
分页 *Paging*

Problem with variable partitioning

A request for reqSize=160K, what should be done?

The total free space is greater than 160, but there is no single free partition larger than 160

It is **memory fragmentation**



Memory compaction:

Merge the free partitions, which requires moving one segment (copying content)

Memory compaction requires a lot of time; if the copying speed is 1M per second, then compacting 1G of memory would take 1000 seconds $\approx$ **17 minutes**



Free partition table

Star. Addr	Size
350K	150K
200K	50K

From continuous to discrete

Ensure the bread is evenly distributed without any leftovers

- Slice the bread, paginate the memory
- For each segment memory request, the system allocates pages one by one to the segment

Question:

Is memory compaction needed at this point?

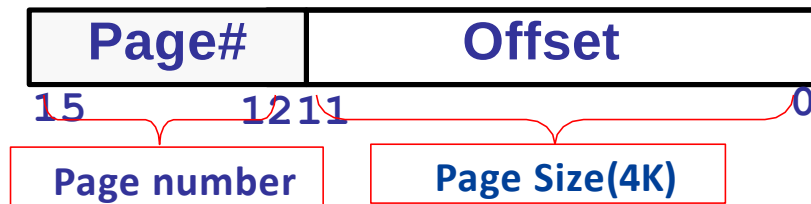
What is the maximum amount of memory waste?

Frame 7	
Frame 6	Seg0 : Page3
Frame 5	Seg0 : Page0
Frame 4	
Frame 3	Seg0 : Page2
Frame 2	
Frame 1	Seg0 : Page1
Frame 0	

Page Table

Ensure the bread is evenly distributed without any leftovers

- Slice the bread, paginate the memory



Frame 7	
Frame 6	Seg0 : Page3
Frame 5	Seg0 : Page0
Frame 4	
Frame 3	Seg0 : Page2
Frame 2	
Frame 1	Seg0 : Page1
Frame 0	

`mov [0x2240], %eax`

Logi. Addr.

0x02

0x240

PT Point

In the PCB

Page#	Frame#	Priv.
0	5	R
1	1	R/W
2	3	R/W
3	6	R

3	240
---	-----

Phys. Addr. : 0x3240

Priv. check

多级页表和快表
Multilevel Paging and TLB

Problem

To improve memory space utilization, the page size should be small.

However, if the page size is too small, the page table becomes larger, consuming more memory.

Page table size is too big

There's always a big difference between demo and practical use!

- The page size is usually 4K, and with a 32-bit address, there can be 1M pages
- 1M page table entries require 4M of memory; With 10 processes running concurrently in the system, 40M of memory would be required

32 bits: [0, 4G]

In reality, most logical addresses are never used

Page#	Frame#	Sign	Valid
0	5	R	1
1	1	R/W	1
2			0
3	6	R	1



Page#	Frame#	Sign
0	5	R
1	1	R/W
3	3	R

Solution 1

Solution 1: Naturally, only the logical pages that are used should have page table entries

But, the page numbers in the page table are not consecutive;
Comparisons and Searches $\log(2^{20})=20$

32bits + 4K + Page# must be consecutive

⇒ 2^{20} Page table entries

⇒ Larger page tables consume memory,

resulting in waste

Solution 2:

The chapter index and the section index

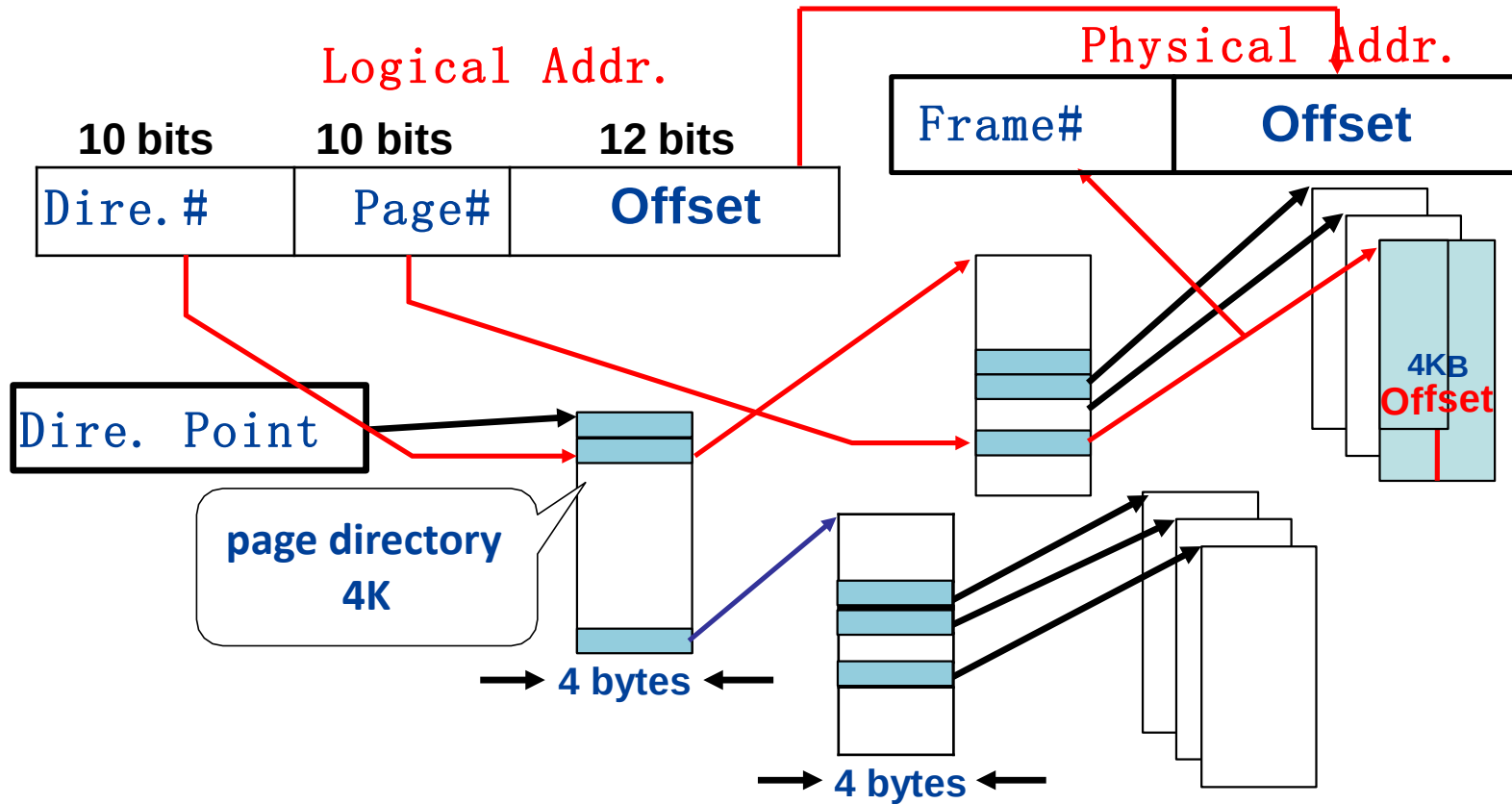
Think by analogy...

Page#	Frame#	Sign
0	5	R
1	1	R/W
3	3	R



Page#	Frame#	Sign	Valid
0	5	R	1
1	1	R/W	1
2			0
3	6	R	1

Solution 2: Multiple-level Page Table



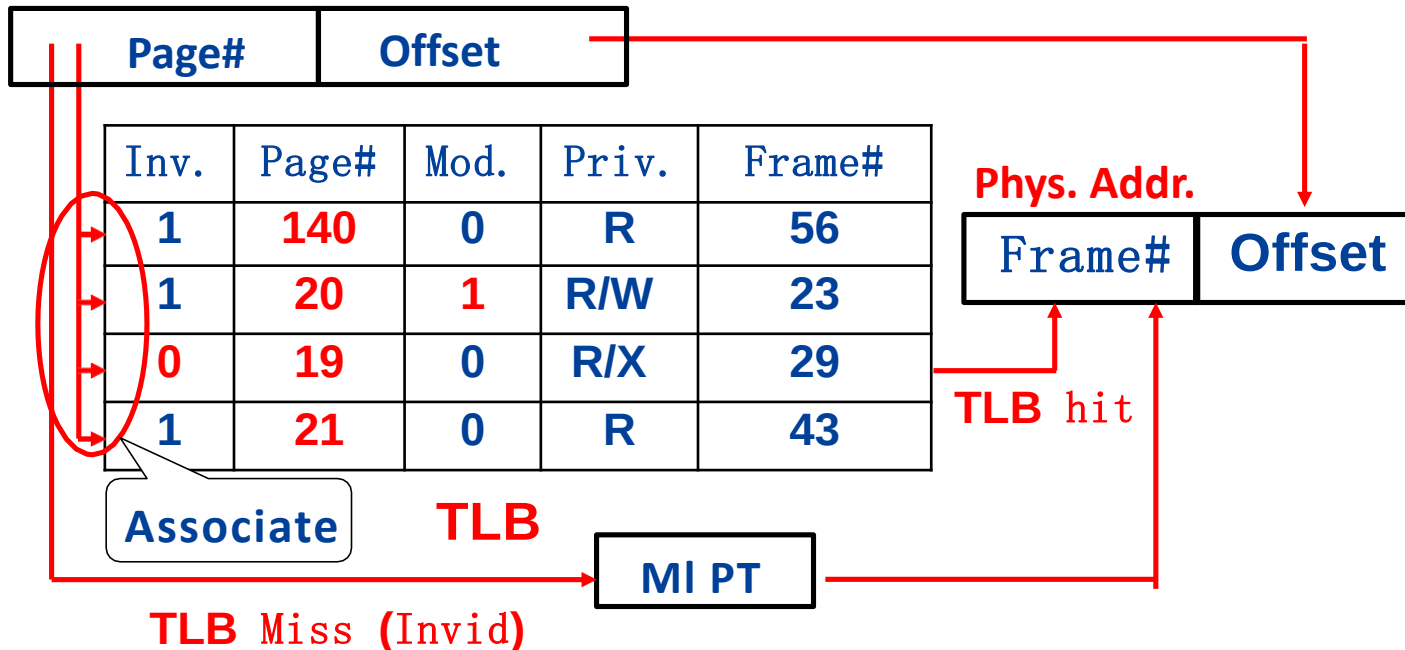
1K directory entries $\times$ 4 bytes per address = 4K, a total of 16K $\ll$ 4M

Another Problem

Multilevel page tables increase the number of memory accesses, especially in 64-bit systems

TLB (Translation Lookaside Buffer) is a set of associative fast storage, akin to registers

Logic. Addr.



TLB is efficient

TLB hit results in high efficiency, while a TLB miss results in reduced efficiency

$$\text{Total Access Time} = \text{HitR} \times (\text{TLB} + \text{MA}) + (1 - \text{HitR}) \times (\text{TLB} + 2\text{MA})$$

Hit Rate

Memory Access Time

TLB Time

$$\text{Total Access Time} = 98\% \times (20\text{ns} + 100\text{ns}) + 2\% \times (20\text{ns} + 200\text{ns}) = 122\text{ns}$$

$$\text{Total Access Time} = 10\% \times (20\text{ns} + 100\text{ns}) + 90\% \times (20\text{ns} + 200\text{ns}) = 210\text{ns}$$

- To truly achieve 'approximately one memory access', the TLB hit rate should be very high
- Larger TLBs are better, but they are expensive; typically, TLB sizes range from [64, 1024]

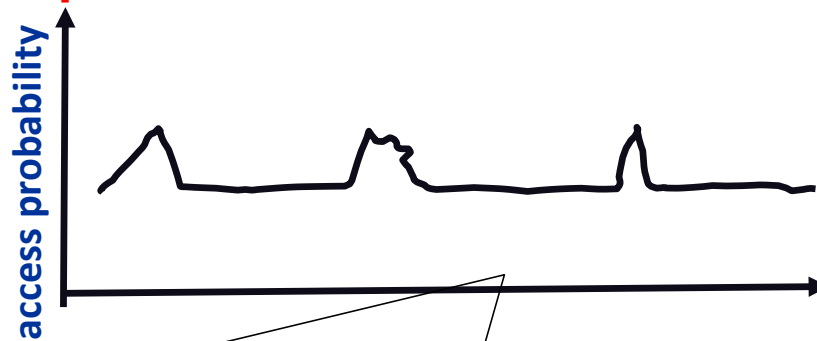
Locality in Space

Compared to 2^{20} pages, 64 is small

Why does the TLB work effectively?

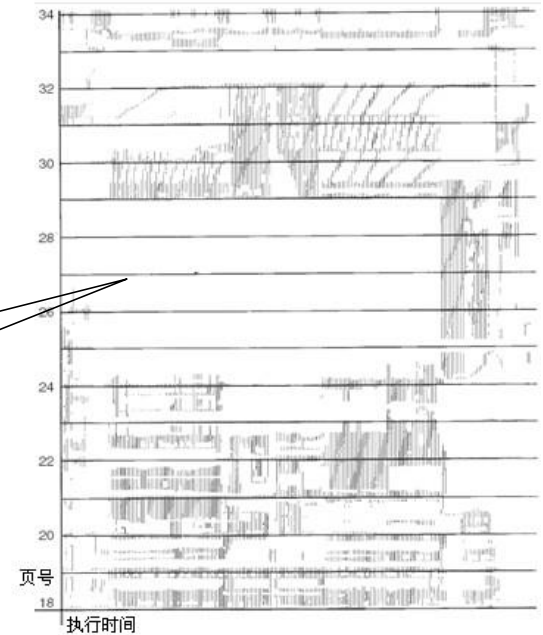
Program address access has **locality**

Locality in Space



a memory
access pattern

Programs often exhibit loops and sequential structures



Computer system design should fully exploit this locality

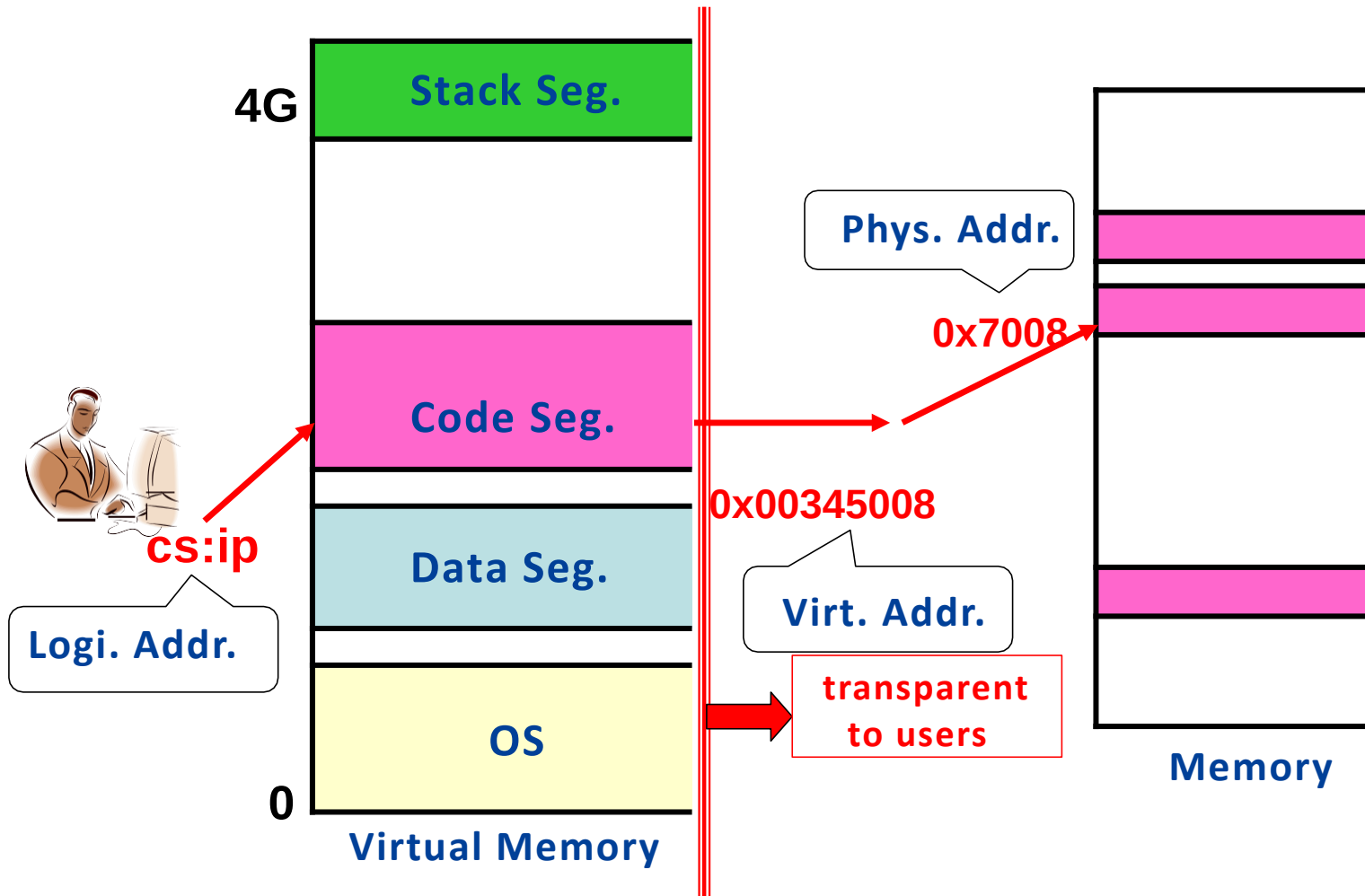
分页 & 分段
Segmentation & Paging

Combination of segmentation and paging:

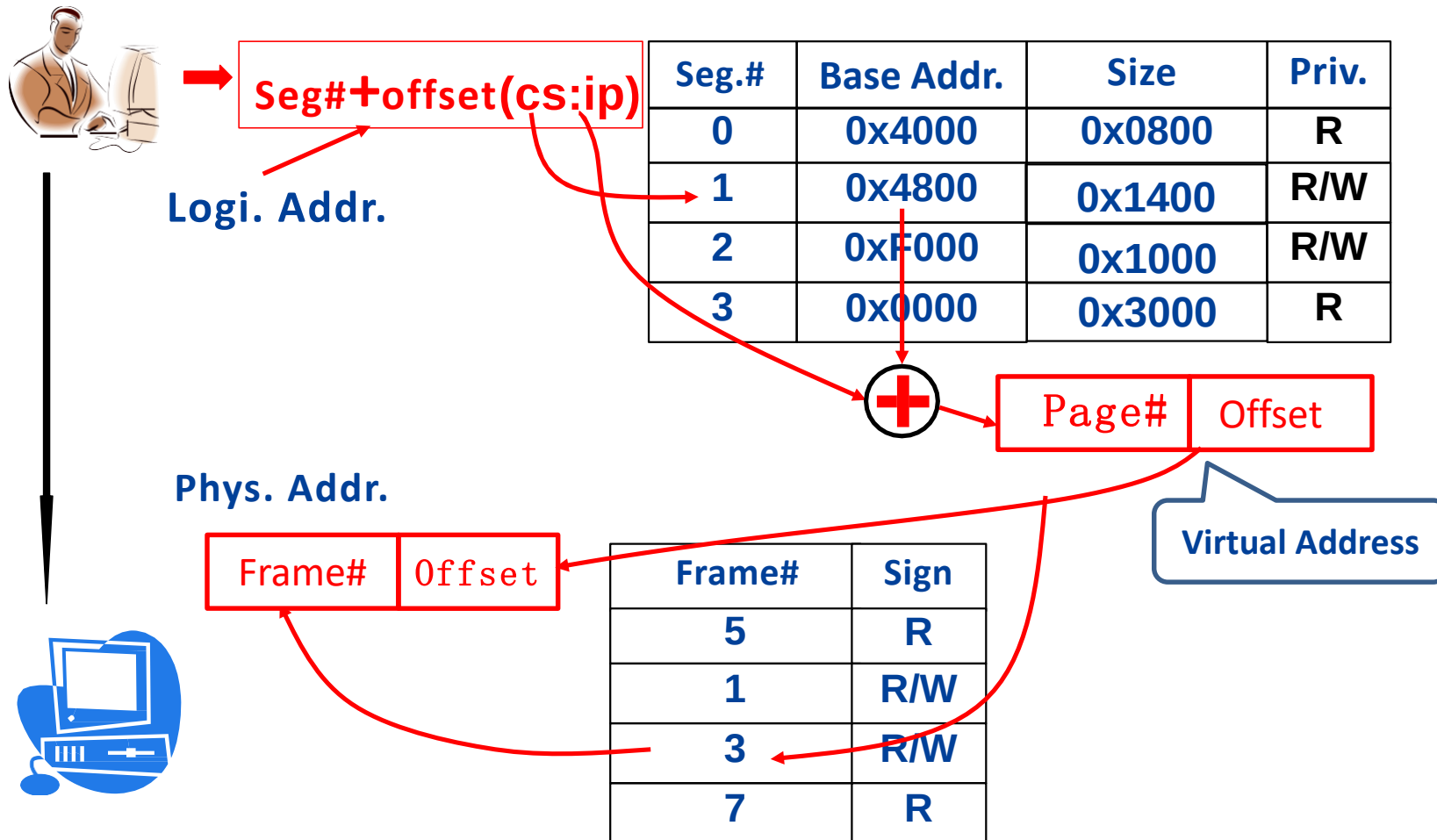
***Programmers prefer using segments,
while physical memory prefers using pages.***

Segment + Page => Virtual Memory

Question: Why is it called virtual memory?



Relocation when segments and pages coexist



Acknowledgement

These slides contain material developed and copyright by:

- Zhijun Li Operating Systems (HIT)